PROGRAMACIÓN WEB CREATIVA II

Taller de Visual Studio Code

Lista de tareas interactiva con JavaScript

HTML, CSS y JavaScript nativo con persistencia en localStorage

Estudiantes	Jhonnier Santiago Giraldo Darién Alonso
Docente	andres.osorioq@ucaldas.edu.co
Asignatura	Programación Web Creativa II
Fecha de entrega	5 de noviembre de 2026

Repositorio: https://github.com/jhonni-png/talleres-programacion-web-creativa-ll (carpeta documentos/taller_lista_de_tareas)

1. Introducción

En este taller se desarrolló una aplicación web para registrar, completar y eliminar tareas. Las tareas se conservan en el navegador mediante localStorage, por lo que siguen disponibles después de recargar la página. Se utilizaron únicamente HTML, CSS y JavaScript nativo, sin librerías ni frameworks, y el proyecto se escribió y probó en Visual Studio Code.

2. Objetivos

- Estructurar una interfaz semántica con HTML (formulario, lista y mensaje de estado).
- Dar estilo a la aplicación con CSS: tarjeta centrada, formulario en una fila y estados visuales para tareas completadas.
- Controlar la lógica con JavaScript: agregar, completar y eliminar tareas, y persistir los datos con localStorage.

3. Estructura del proyecto

Archivo	Responsabilidad
`index.html`	Estructura de la página: formulario, lista vacía (ul), mensaje de lista vacía y enlaces a styles.css y script.js.
`styles.css`	Apariencia: tarjeta centrada sobre fondo claro, tareas separadas por líneas, texto tachado y atenuado para las completadas y botón rojo para eliminar.
`script.js`	Lógica: carga desde localStorage, dibuja la lista, agrega, marca como completada, elimina y guarda cada cambio.

4. Código HTML · index.html

Define el formulario con un campo de texto (limitado a 80 caracteres y obligatorio) y el botón «Agregar», la lista vacía donde JavaScript dibuja las tareas y el mensaje que se muestra cuando no hay tareas.

```
<!DOCTYPE html>
<!--
  Programación Web Creativa II · Taller: Lista de tareas interactiva con JavaScript
  Autores: Jhonnier Santiago Giraldo y Darién Alonso
-->
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lista de tareas</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main class="app">
    <h1>Lista de tareas</h1>

    <form id="formTarea" class="formulario">
      <input type="text" id="inputTarea" placeholder="Escribe una tarea..."
        maxlength="80" autocomplete="off" required>
      <button type="submit">Agregar</button>
    </form>

    <ul id="listaTareas"></ul>
    <p id="mensajeVacio">No hay tareas pendientes. ¡Agrega la primera!</p>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

5. Código CSS · styles.css

Aplica un diseño de tarjeta centrada con Flexbox. La clase .completada tacha y atenúa el texto, y la clase .eliminar da al botón de borrar un color rojo que lo diferencia de la acción principal.

```
/* Programación Web Creativa II · Taller: Lista de tareas
   Autores: Jhonnier Santiago Giraldo y Darién Alonso */

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  padding: 48px 16px;
  font-family: "Segoe UI", Arial, sans-serif;
  background: #eef2f7;
}

.app {
  width: min(100%, 480px);
  padding: 28px 24px;
  background: #ffffff;
```

```
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(31, 58, 95, 0.12);
}

h1 { margin-bottom: 20px; font-size: 1.5rem; color: #1f3a5f; }
.formulario { display: flex; gap: 8px; margin-bottom: 20px; }

input[type="text"] {
  flex: 1;
  padding: 10px 12px;
  font-size: 1rem;
  border: 2px solid #d5dde8;
  border-radius: 8px;
  outline: none;
}

input[type="text"]:focus { border-color: #0f766e; }

button {
  padding: 10px 16px;
  font-weight: 600;
  color: #ffffff;
  background: #0f766e;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

ul { list-style: none; }

li {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 4px;
  border-bottom: 1px solid #e5eaf1;
}

li span { flex: 1; word-break: break-word; }
li.completada span { color: #94a3b8; text-decoration: line-through; }
.eliminar { padding: 6px 10px; font-size: 0.8rem; background: #c53030; }
#mensajeVacio { text-align: center; color: #64748b; }
```

6. Código JavaScript · script.js

Mantiene un arreglo de objetos {id, texto, completada}. Cada cambio pasa por actualizar(), que guarda en localStorage y redibuja la lista, de modo que el almacenamiento y la pantalla nunca se desincronizan.

```
/* Programación Web Creativa II · Taller: Lista de tareas
   Autores: Jhonnier Santiago Giraldo y Darién Alonso */

const form = document.getElementById("formTarea");
const input = document.getElementById("inputTarea");
const lista = document.getElementById("listaTareas");
const mensajeVacio = document.getElementById("mensajeVacio");
const CLAVE = "tareas"; // nombre con el que se guarda en localStorage

// Recupera las tareas guardadas; si no hay datos válidos, devuelve []
function cargarTareas() {
  try { return JSON.parse(localStorage.getItem(CLAVE)) || []; }
  catch { return []; }
}
let tareas = cargarTareas();
```

```
// Guarda en localStorage y vuelve a dibujar la lista
function actualizar() {
  localStorage.setItem(CLAVE, JSON.stringify(tareas));
  mostrarTareas();
}

function mostrarTareas() {
  lista.innerHTML = "";
  mensajeVacio.hidden = tareas.length > 0; // visible solo si no hay tareas

  tareas.forEach((tarea) => {
    const item = document.createElement("li");
    item.className = tarea.completada ? "completada" : "";

    const casilla = document.createElement("input");
    casilla.type = "checkbox";
    casilla.checked = tarea.completada;
    casilla.onchange = () => { tarea.completada = !tarea.completada; actualizar(); };

    const texto = document.createElement("span");
    texto.textContent = tarea.texto; // textContent evita insertar HTML

    const borrar = document.createElement("button");
    borrar.className = "eliminar";
    borrar.textContent = "Eliminar";
    borrar.onclick = () => {
      tareas = tareas.filter((t) => t.id !== tarea.id);
      actualizar();
    };

    item.append(casilla, texto, borrar);
    lista.appendChild(item);
  });
}

form.addEventListener("submit", (evento) => {
  evento.preventDefault();
  const texto = input.value.trim();
  if (texto === "") return;
  tareas.push({ id: Date.now(), texto, completada: false });
  actualizar();
  form.reset();
  input.focus();
});

mostrarTareas(); // muestra las tareas al cargar la página
```

7. Explicación del funcionamiento

- **cargarTareas()** lee la clave «tareas» de localStorage dentro de un try/catch; si el dato no existe o está corrupto, devuelve un arreglo vacío y la aplicación no se rompe.
- **actualizar()** guarda el arreglo como JSON y vuelve a dibujar la lista; es el único punto de salida de cualquier cambio.
- **mostrarTareas()** limpia la lista, muestra u oculta el mensaje vacío con la propiedad hidden y crea por cada tarea un li con casilla, texto y botón. Usa textContent, así que lo escrito por el usuario nunca se interpreta como HTML (previene XSS).
- **Evento submit** evita el envío del formulario, recorta espacios con trim(), ignora textos vacíos, crea la tarea con id = Date.now() y deja el campo limpio con el foco listo para la siguiente.

- **Completar y eliminar:** la casilla invierte tarea.completada y el botón rojo elimina la tarea por su id con filter().

8. Pruebas y evidencias

El proyecto se abrió en un navegador (Chromium) y se verificó cada funcionalidad. Resultados de las 12 comprobaciones realizadas, sin errores en la consola:

#	Prueba	Resultado
1	Con la lista vacía se muestra el mensaje «No hay tareas pendientes».	Correcto ✔️
2	Se agregan tres tareas y aparecen en la lista.	Correcto ✔️
3	Al haber tareas, el mensaje de lista vacía se oculta.	Correcto ✔️
4	Un texto con etiquetas HTML se muestra como texto plano, sin ejecutarse.	Correcto ✔️
5	Tras agregar, el campo queda vacío y con el foco.	Correcto ✔️
6	Marcar la casilla tacha y atenúa la tarea.	Correcto ✔️
7	Al recargar la página, las tareas y su estado se conservan.	Correcto ✔️
8	El botón Eliminar quita la tarea seleccionada.	Correcto ✔️
9	La eliminación persiste después de recargar.	Correcto ✔️
10	Un texto con solo espacios no agrega ninguna tarea.	Correcto ✔️
11	Al eliminar todas las tareas vuelve a mostrarse el mensaje vacío.	Correcto ✔️
12	Un valor corrupto en localStorage no rompe la aplicación.	Correcto ✔️

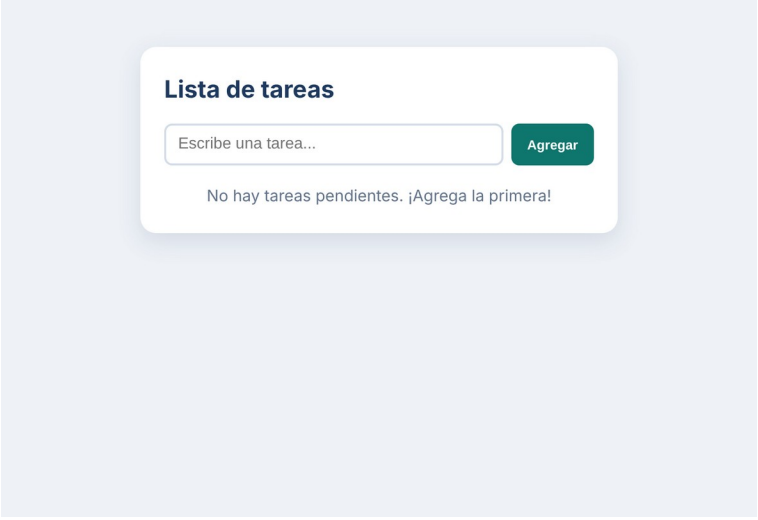


Figura 1. Estado inicial: lista vacía con su mensaje.



Figura 2. Tareas agregadas (el texto con etiquetas HTML se muestra tal cual).



Figura 3. Una tarea marcada como completada: texto tachado y atenuado.



Figura 4. Tras recargar la página, las tareas siguen en pantalla gracias a localStorage.

9. Conclusiones

- Separar estructura (HTML), presentación (CSS) y comportamiento (JavaScript) hace el proyecto fácil de leer y mantener.
- Centralizar el guardado y el redibujado en una sola función evita inconsistencias entre los datos y la interfaz.
- localStorage permite persistencia sencilla en el navegador; el manejo de errores con try/catch lo hace robusto ante datos inválidos.
- Usar textContent en lugar de innerHTML para el texto del usuario es una práctica básica de seguridad frente a la inyección de código.

10. Cómo ejecutar el proyecto

Abrir la carpeta taller_lista_de_tareas en Visual Studio Code y abrir index.html en el navegador (o usar la extensión Live Server). No requiere instalación de dependencias.